



Real-time unsupervised video object detection on the edge

Paula Ruiz-Barroso*, Francisco M. Castro, Nicolás Guil

Department of Computer Architecture, University of Málaga, Spain

ARTICLE INFO

Keywords:

Embedded system
Unsupervised object detection
Model quantization
Knowledge distillation

ABSTRACT

Object detection in video is an essential computer vision task. Consequently, many efforts have been devoted to developing precise and fast deep-learning models for this task. These models are commonly deployed on discrete and powerful GPU devices to meet both frame rate performance and detection accuracy requirements. Furthermore, model training is usually performed in a strongly supervised way so that samples must be previously labelled by humans using a slow and costly process. In this paper, we develop a real-time implementation for unsupervised object detection in video employing a low-power device. We improve typical approaches for object detection using information supplied by optical flow to detect moving objects. Besides, we use an unsupervised clustering algorithm to group similar detections that avoid manual object labelling. Finally, we propose a methodology to optimize the deployment of our resulting framework on an embedded heterogeneous platform. Thus, we illustrate how all the computational resources of a Jetson AGX Xavier (CPU, GPU, and DLAs) can be used to fulfil frame rate, accuracy, and energy consumption requirements. Three different data representations (FP32, FP16 and INT8) are studied for the pipeline networks in order to evaluate the impact of all of them in our pipeline. Obtained results show that our proposed optimizations can improve up to 23.6× energy consumption and 32.2× execution time with respect to the non-optimized pipeline without penalizing the original mAP (59.44). This computational complexity reduction is achieved through knowledge distillation, using FP16 data precision, and deploying concurrent tasks in different computing units.

1. Introduction

Nowadays, automatic video surveillance systems typically implement object detection techniques as an initial phase before performing higher semantic tasks such as event detection or human activity recognition. Most successful approaches for object detection have been developed using Deep Learning (DL) models or networks like CNNs [1,2] or Transformers [3,4]. These models are built using a supervised learning approach where the images of object classes together with their labelling must be available before training. Typically, these models have high computational complexity and must be run in accelerators, like discrete GPUs, to fulfil throughput requirements.

Simultaneously, embedded system vendors have significantly increased the computational power of their devices by implementing heterogeneous architectures that include hardware accelerators [5–7]. This way, these embedded platforms can execute complex DL models with a reduced power consumption budget. However, original models must be optimized before deploying them on these architectures to fulfil both computation time and power consumption requirements while keeping the original model's accuracy.

In Fig. 1 we show the typical input, i.e. a set of consecutive RGB frames recorded by a camera, that is processed by the embedded system

for object detection applications. The output is a set of coordinates that describes rectangular regions at each frame (depicted in different colours in the Figure) with labels indicating the object class detected at each region.

Several works regarding the deployment of object detection solutions on edge devices have been published in the last few years [8,9]. However, DL models employed in those works present an important weakness to be pervasively deployed in video surveillance systems, as they lack the capacity for domain adaption. Thus, if a model is trained, for instance, to detect objects on the road, it is not going to work properly when deployed on the airport runway as new unknown object classes appear. This situation can be solved by retraining the model with new labelled data, if available, but this process is time- and energy-consuming and, consequently, expensive. To solve the model domain adaptation we propose a framework that employs an unsupervised approach for object localization in videos, which is composed of two region proposals: one based on RGB images and another one based on optical flow maps which helps us to detect small and moving objects, followed by an unsupervised stage to group similar objects [10]. This new framework considerably increases the computational complexity of the current object detector, which makes challenging its deployment

* Correspondence to: Bulevar Louis Pasteur, 35, Office 2.3.8a, 29071, Malaga, Spain.
E-mail address: pruizb@uma.es (P. Ruiz-Barroso).



Fig. 1. Edge computing: video object detection on an embedded device. The input is a set of video frames. The output is a set of bounding boxes with an objectness score given by a model implemented on an embedded device.

in an embedded system, but it helps to deal with the model adaptation problem.

Thus, in contrast to previous works that propose unsupervised pipelines requiring several expensive GPU accelerators [11], supervised approaches that do not use heterogeneous deployments to take advantage of all available processors [8] or do not study the impact of the numerical precision [12], we propose an unsupervised object detection approach using all available computing resources of the embedded architecture (CPU, GPU, or specific accelerators) together with a thorough study of several arithmetic data representations (FP32, FP16, and INT8).

Concretely, in our experiments, we test our proposal using a live RGB camera continuously recording the *apron area* (the area where airplanes park to load passengers and luggage) of the Gdansk Airport.¹ Results show the robustness of our approach, especially with small regions or classes with changes in shape such as persons, and can run using an embedded system at an adequate frame rate to obtain good results in a video surveillance application.

Consequently, the main contributions of this paper are: (i) a complete pipeline for unsupervised object detection in videos; (ii) a combination of two complementary region proposals algorithms specially designed for video sequences; (iii) an optimized pipeline for embedded systems after exploring the use of all available computing units and several data quantization; and, (iv) a thorough experimental study to validate the proposed framework and different optimizations.

The rest of the paper is organized as follows. After presenting some background in Section 2, Section 3 summarizes the related work. Section 4 introduces our unsupervised object detection pipeline. Hardware description and our pipeline optimizations are explained in Section 5. Then, in Section 6 we define the experimental setup and discuss the results. Finally, Section 7 concludes the paper.

2. Background

2.1. Deep learning on the edge

Over the last few years, the use of embedded platforms to deploy deep learning applications has become more important, powered by IoT [13], health care systems [14] or autonomous driving [15]. In order to increase inference throughput and reduce energy consumption on embedded devices, many contributions have been made by applying techniques such as quantization [16], which reduces computation complexity using simpler data representation, or pruning [17], that removes redundant parameters or neurons that do not contribute significantly to the accuracy results.

Current embedded systems employ heterogeneous architectures where CPU cores share main memory with other accelerators, such as GPUs. In Table 1, various edge architectures, which include CPU and GPU cores, have been shown jointly with their computational resources and operating frequency values. As can be seen, each system provides a different performance and energy consumption, which makes the choice of the board very important. Moreover, there exist software

development kits with the aim of optimizing deep learning models such as TensorRT [18] or TensorFlow Lite [19] for GPU and CPU, respectively. For NVIDIA devices, TensorRT contains a deep learning inference optimizer and a runtime that allows obtaining low latency and high throughput for inference applications using quantization, kernel auto-tuning, memory reuse and layer fusion. Besides, parallelization is employed to maximize the throughput in different accelerators such as GPU and deep learning accelerators (DLAs) [20]. Object detection networks and segmentation approaches have also been optimized with TensorRT [21,22]. Focusing on TensorFlow Lite, it also applies optimizations such as quantization or pruning and is commonly used on two different devices: microcontrollers [23] and smartphones [24].

In this work, we have deployed our proposal on an NVIDIA Jetson AGX Xavier (see Table 1) since, according to previous works [25–27], it is currently one of the most powerful devices for embedded computer vision applications as seen in Table 1, with a reasonable power consumption due to it was designed to be used as a low power consumption system. It has a wide range of operating frequencies for the CPU, the GPU and the eMMC. Moreover, the CPU cores can be disconnected individually. These characteristics allow us a high degree of hardware customization and, consequently, performance and power consumption.

2.2. Object detection

Object detection methods based on deep learning are composed of two main blocks as shown in Fig. 2: the backbone, which produces high-level descriptors from the input image, and the detection head, which uses the output of the backbone to localize and classify the different objects appearing in the input image. Traditionally, the backbone was a pretrained CNN model [28–30] for image classification to ensure obtaining good image features. However, with the advent of attention mechanisms [31], visual transformers (ViT) [32] can perform this task [33,34]. Finally, the detection head is common for all models and consists of a regressor to localize the objects and a classifier to identify the class of the localized area.

From a behavioural point of view, detection models can be broadly categorized into two main approaches: two-stage detectors and one-stage detectors. Two-stage detectors follow a two-step process: region proposal generation and object classification. Using the features extracted with a backbone from the input image, region proposal generation aims to find potential object locations, also known as region proposals. It typically involves techniques like selective search or region proposal, which propose a set of candidate bounding boxes likely to contain objects. Then a classifier is used to assign object labels. A regressor can be optionally used to refine the bounding box predictions. Prominent examples of two-stage detectors include Faster R-CNN [1], a region-based Convolutional Neural Network and Mask R-CNN [28], an extension of Faster R-CNN that also provides instance-level segmentation. On the other hand, in one-stage detectors, region proposal generation and object classification are done directly at the same time. Therefore, one-stage detectors work without pregenerated region proposals, a feature extractor using a backbone processes the images, and then these features are used for classification and regression to obtain bounding box coordinates. YOLO (You Only Look Once) models [29] and SSD (Single Shot MultiBox Detector) [30] are examples of well-known one-stage detectors.

Typically, one-stage detectors obtain better throughput, measured in frames per second, than two-stage ones, but lower accuracy.

2.3. Learning paradigms

The learning process of DL models can be divided into three main categories [35]: supervised learning, semi-supervised learning and unsupervised learning. Supervised learning is the standard paradigm since it obtains models with the best performance. However, this kind of

¹ <http://www.airport.gdansk.pl/airport/kamery-internetowe>

Table 1

Main characteristics of modern embedded computing boards with heterogeneous architecture. The first columns provide detailed specifications of the CPU, GPU, and RAM, along with information on the presence or absence of dedicated Deep Learning Accelerators (DLAs). The last two columns show theoretical FLOPS (floating point operations per second) or OPS (operations per second) and available power modes, respectively.

Board Name	CPU	GPU	RAM (GB)	DLAs	AI Performance	Power modes (W)
Raspberry Pi 4	4x ARM Cortex A72@1.5 GHz	VideoCore VI@500MHz	8	–	32 GFLOPS	8
Raspberry Pi 5	4x ARM Cortex A76@2.4 GHz	VideoCore VII@800MHz	8	–	52 GFLOPS	12
Google Coral	4xARM Cortex A53@1.5GHz + Cortex-M4F@168MHz	2D/3D Vivante GC7000 Lite@0.8 GHz	4	Google TPU	4 TOPS	2
Jetson Nano	4x A57@1.43 GHz	128-core NVIDIA Maxwell@921 MHz	4	–	472 GFLOPS	5 10
Jetson AGX Xavier	8xNVIDIA Carmel ARM v8.2@2.2 GHz	512-core NVIDIA Volta 64 Tensor Cores@1.37 GHz	32	2xNVDLA	32 TOPS	10 15 30
Jetson AGX Orin	8xNVIDIA ARM Cortex A78AE v8.2@2.2 GHz	1792-core NVIDIA Ampere 56 Tensor Cores@930MHz	64	2xNVDLA v2	200 TOPS	15-40

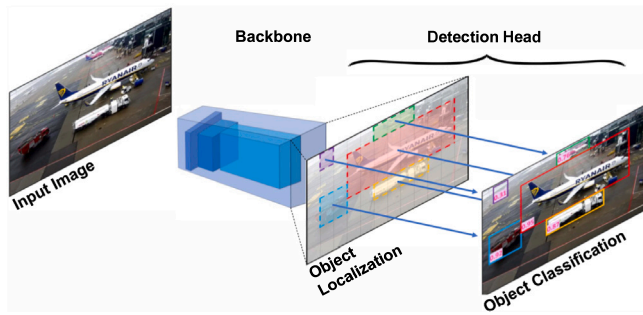


Fig. 2. Object detection method based on deep learning. The backbone produces high-level descriptors from the input image. Object localization and object classification blocks make up the detection head which uses the backbone output to localize and classify the objects from the input image, respectively.

learning paradigm requires a huge labelled training set with high variability between samples. This requirement imposes a huge limitation in the application of this learning paradigm in tasks where the amount of labelled data is reduced or the labelling process is tedious and expensive.

On the opposite side, we found unsupervised learning, which requires a huge training set but labels are not necessary. This way, raw data extracted from online resources can be used without human supervision. However, models trained with this paradigm focus on describing the input samples in a higher dimensionality producing similar outputs from similar inputs. Thus, these models work as a kind of clustering algorithm that can be used as pretrained models for other downstream tasks [36,37] requiring less labelled data and training time since the training process does not start from scratch.

Finally, in the middle point, semi-supervised learning combines both paradigms using a small set of labelled samples together with a huge set of unlabelled samples. Usually, the unlabelled samples come from the same domain as the labelled data, however, they can be from completely different domains and the models should deal with that situation during training. In this sense, many techniques [38,39] have been proposed to select or even generate auxiliary samples that could improve the learning process of the models. In terms of performance, semi-supervised models usually provide lower results than supervised models but, like with unsupervised models, they can be used as pretrained models for other downstream tasks.

3. Related work

In this section, we present the state of the art by focusing on two key areas: edge computing for object detection and unsupervised and weakly-supervised approaches. A summary of the comparison between these works and our proposal is provided in Table 2.

3.1. Edge computing for object detection

Object detection methods can be deployed on edge systems but, typically, some optimization techniques like model quantization [16], pruning [17], and model compression [40] are applied to reduce the model's size and computational and energy consumption requirements. However, some authors do not apply any of these optimization techniques and are more focused on changing the layers or backbone of the model. Ren et al. [41] present an edge computing-based object detection architecture composed of three layers: the IoT (Internet of Things) device layer, the edge server layer, and the cloud layer. Real-time data is captured and compressed by IoT devices and transmitted to the edge servers for ROIs (Regions of interest) detection by using a Faster R-CNN model. The cloud server receives unknown objects or low-scored objects found by distributed edge servers. Moreover, the cloud server has global knowledge of the edge servers and updates inference models. Mao et al. [12] have designed a lightweight network based on YOLOv3 called Mini-YOLOv3. They have reduced the number of parameters of the network by using depth separable convolutions and pointwise group convolutions. To avoid the drop in accuracy, they added a Multi-Scale Feature Pyramid network based on a simple U-shaped structure to improve the performance of multi-scale object detection. Chiu et al. [8] also introduce a lightweight object detection model based on Mobilenet-v2, called Mobilenet-SSDv2. They use Mobilenet instead of VGG16 as the backbone of the SSD detector in order to reduce latency. Then they merge a Feature Pyramid Network with the proposed object detection model to improve detection accuracy.

On the other hand, some authors use techniques such as quantization to reduce the computational requirements of edge devices. Matsubara et al. [9] divide the object detector model into a head and a tail, thus, the first layers of the model (head) are deployed on the edge system and the final layers (tail) are computed in the cloud. A quantized bottleneck layer is added at the end of the edge model to reduce the bandwidth required by the transmitted features. Other authors examine the use of specialized frameworks to deploy optimized deep learning models such as Choe et al. [42].

Finally, transformer-based models, which have a large number of parameters and layers, are not usually deployed in embedded systems due to their huge execution time and energy consumption [32]. The resource consumption of this kind of model makes it impossible to deploy a complete pipeline with several models like the proposed approach in this work. Thus, we propose the deployment of an efficient traditional ROI detector together with an unsupervised classifier that allows the identification of new types of objects.

3.2. Unsupervised and weakly-supervised approaches

In recent years, unsupervised [10], semi-supervised [43] and weakly supervised [11] approaches have gained importance due to the lack

Table 2

Related work. Comparison of related works based on the learning paradigm, model acceleration and the use of edge computing, highlighting the differences with our proposal.

Related work	Learning paradigm	Model acceleration	Edge computing
[41]	Supervised	×	×
[9,12]	Supervised	✓	×
[8,42]	Supervised	✓	✓
[11,44–47]	Unsupervised	×	×
[48–50]	Weakly-supervised	×	×
Our work	Unsupervised	✓	✓

of labelled data in many real-life problems. On the one hand, some works are focused on unsupervised learning for object localization. Gudovskiy et al. [44] present CFLOW-AD model, that is based on a conditional normalizing flow framework adopted for anomaly detection and localization using an encoder and generative decoders. Zimmerer et al. [45] complement localization with variational auto-encoders. Moreover, some proposals are related to low-cost devices. In this field, Li et al. [46] propose a deep-reinforcement-learning-based unsupervised wireless-localization method. Chen et al. [47] present an unsupervised learning strategy to train the fingerprint roaming model using unlabelled data which does not incur labour-intensive data collection and annotation costs. On the other hand, several contributions have been proposed in the area of weakly supervised learning related to classification problems. Wang et al. [48] developed a weakly-supervised framework for COVID-19 classification and lesion localization. Wang et al. [49] proposed a weakly-supervised classification method based on cross-slide contrastive learning. Liu et al. [50] present a weakly-supervised transfer learning framework to classify multi-temporal remote-sensing images with only one labelled image.

In our work, we propose an unsupervised approach for object localization and classification.

4. Unsupervised object detection proposal

Our goal is to implement a real-time² unsupervised object detection pipeline on videos using an embedded device. Thus, in this section, we present our pipeline, as shown in Fig. 3, which is composed of the following stages: (1) object localization using a region proposal network (RPN) together with a region proposal based on optical flow (OFRP) maps, (2) Non-maxima suppression (NMS) to combine objects from RPN and OFRP, (3) encoding of localized objects using a Region Encoding Network (REN), (4) a clustering algorithm that groups similar regions based on the obtained features. Notice that previous implementations for video object detection in embedded systems employ trained models using just the RPN module [9,42], which impedes the adaptability of the detection model to different scenarios. In this work, we add an unsupervised process to increase implementation flexibility. It requires the introduction of a new clustering module. In addition, the object localization accuracy of previous implementations is improved using optical flow modality as input, requiring the implementation of an Optical Flow Region Proposal, a Non-maxima suppression module and a Region Encoder.

4.1. Input data

We use video frames obtained from an RGB camera recording the apron area (the area where aeroplanes are parked, unloaded, loaded, boarded, refuelled or maintained) at the Gdansk Airport (Poland). This camera is publicly available online.

² Note that in our application, 250 ms (4 FPS) would be sufficient to detect moving objects due to the fact that the scene does not change quickly enough to lose detections.

4.2. Region proposal network (RPN)

A Region Proposal Network, or RPN, is composed of a classifier and a regressor, as can be observed in Fig. 2. The classifier predicts the probability that a proposed region contains a target object and the regressor returns the coordinates of the proposed regions. The RPN will be able to detect mostly the largest objects present in the foreground because these detections are commonly used for training these widespread networks. The bounding boxes acquired in this stage are fed into the next stage of the pipeline, *i.e.* in the non-maxima suppression stage, which is detailed in Section 4.4.

4.3. Optical flow region proposal (OFRP)

Region proposal based on OF is obtained from optical flow maps in order to localize mainly small objects, which are less likely to be detected by the previously commented RPN. OF maps F_t are computed every 5 frames to acquire better OF results for the smallest objects. To remove OF noise generated by changes in the scenario (e.g illumination, shadows, etc.), all positions whose OF vector components (x and y) are smaller than a threshold T_F are set to 0. An example of the obtained optical flow is shown in Fig. 4 using different colours that represent the intensity of optical flow in absolute value. Then, contours that contain objects are found using the algorithm proposed by Suzuki et al. [51] and eliminated if their area is smaller than a threshold T_A . To prevent intermittent localizations, we track every F_T region to the next OF map F_{t+1} using the mean OF components of this region. If it is missed in F_{t+1} , we remove the original detection from F_t . Finally, a bounding box is created around every contour to obtain object localization similarly to using RPN in the previous stage. These regions are fed into the next stage of the pipeline, *i.e.* in the non-maxima suppression stage, which is detailed in Section 4.4.

4.4. Non-maxima suppression (NMS)

To avoid the repetition of the same objects localized by RPN and OFRP at the same time, we combine both localizations. To do that, we compute the intersection over union (IoU) metric between these two proposed regions. If IoU is bigger than a threshold T_I (Eq. (1)) and the aspect ratio of the biggest region divided by the smallest one is bigger than a threshold T_{AR} (Eq. (2)), *i.e.* the aspect ratio of both regions is similar, both regions are overlapped and we only keep the bounding box proposed by RPN due to the fact that it is more accurate than the bounding box proposed using OFRP. Furthermore, we apply non-maxima suppression to each proposal algorithm to remove overlapped regions whose IoU is greater than the same threshold T_I . Note that the objects that not meet the previous equations requirements are preserved. All these final regions are fed into the next stage of the pipeline, *i.e.* in the Region Encoding Network, which is detailed in Section 4.5.

$$IoU = \frac{Area\ of\ Overlap}{Area\ of\ Union} > T_I \quad (1)$$

$$\frac{\max\left(\frac{height_{RPN}}{width_{RPN}}, \frac{height_{OFRP}}{width_{OFRP}}\right)}{\min\left(\frac{height_{RPN}}{width_{RPN}}, \frac{height_{OFRP}}{width_{OFRP}}\right)} > T_{AR} \quad (2)$$

4.5. Region encoding network (REN)

To group similar regions into clusters, we need to describe the visual information contained in each bounding box obtained in the previous stage and then introduce this information into the clustering algorithm. Notice that the ROI content descriptor calculated by the RPN is not used as also ROIs detected by OFRP must be encoded, *i.e.* all ROIs must be encoded with the same model.

Finally, the features of these localized objects are obtained from the last layer of the pretrained model and we use them as input for the next stage of our pipeline, *i.e.* in the clustering algorithm, which is detailed in Section 4.6.

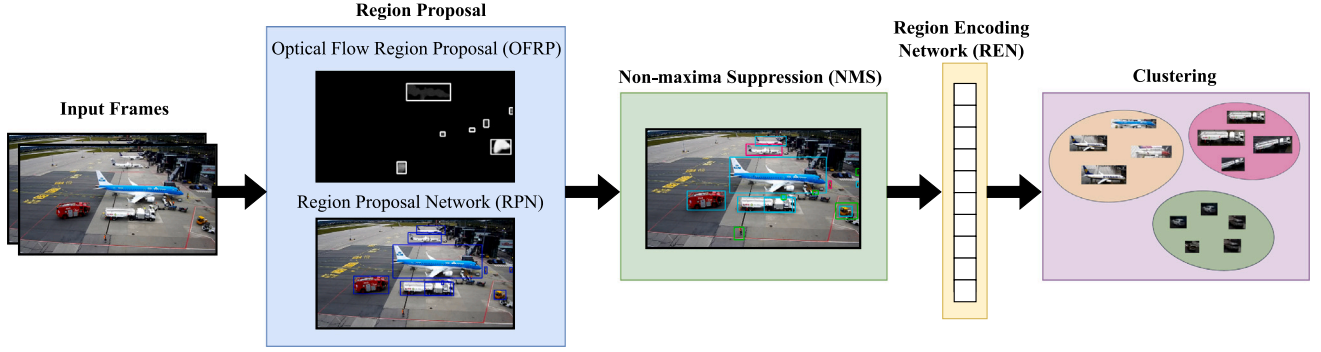


Fig. 3. Object localization and classification pipeline. The input is a subset of RGB video frames. ROIs are obtained from the optical flow region proposal (*OFRP*) and the region proposal network (*RPN*). Non-maxima suppression (*NMS*) is applied to remove overlapped localizations. Regions proposed by *OFRP*, *RPN* and by both algorithms are marked in green, blue and pink, respectively. Features from ROIs are extracted using a region encoding network (*REN*). A clustering algorithm groups similar features.

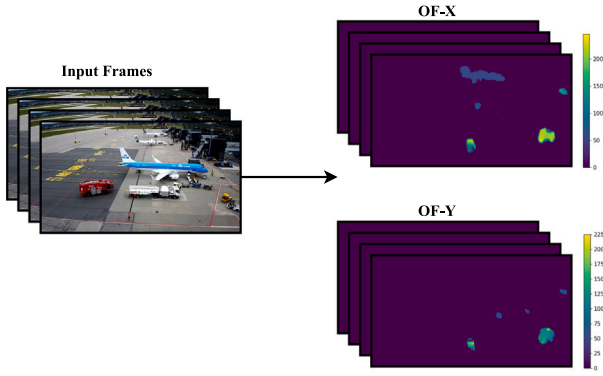


Fig. 4. Optical Flow. Optical Flow vector components (x and y) are represented as different images. Note that colours represent the intensity of optical flow in absolute value. This representation is used to generate bounding boxes enabling localization of moving objects.

4.6. Clustering

The final objective of this stage is, for each ROI, to find the closest cluster of similar regions to assign to each ROI the soft label that identifies the selected cluster. These soft labels indicate the probability that a region is part of a certain cluster. Note that we are not using the predictions made by *REN* because we would only be capable of correctly predicting those objects previously seen by *REN*. Thus, using our clustering step, we are able to continuously create new clusters formed by new classes that have not ever been seen before. In order to improve the clustering accuracy and accelerate the computation of this stage, we use a dimensionality reduction algorithm since it has been shown that its use can drastically improve both metrics for well-known clustering algorithms [52].

5. Deployment on the edge platform

In this section, we study the deployment of our proposed unsupervised pipeline for object detection on a heterogeneous embedded device by employing different optimization techniques. We also explore the use of all available computing resources of the target hardware with the aim of obtaining a suitable throughput that guarantees real-time operation and good accuracy. In the following subsections, we describe the available hardware of the embedded device, focusing on the processing units and the supported numerical precision. Next, we show the deployment decisions that we have carried out to take advantage of the computing resources of the available processing units.

5.1. Hardware description

We have employed a NVIDIA Jetson AGX Xavier, which is a heterogeneous embedded device that includes a multicore CPU, a general-purpose accelerator (GPU), and two specific accelerators (DLAs), namely DLA-0 and DLA-1. Each computing hardware supports different numerical precision to further reduce both computation time and energy consumption. Following we describe the capabilities and limitations of this hardware.

5.1.1. Computing hardware

NVIDIA Jetson AGX Xavier, as it can be observed in Table 1, includes an octa-core NVIDIA Carmel ARM v8.2 CPU and a NVIDIA Volta with 512 cores and 64 tensor cores GPU. Although some authors Jia et al. [53], Kim et al. [54] obtained better throughput by deploying deep learning models on CPU in other types of heterogeneous architectures where the computational power of GPU and CPU is balanced, this is not the case for NVIDIA devices as proved in [25]. Thus, in this work, the CPU is not used for complete or partial deployment of deep learning models because its computational power in the NVIDIA Xavier is much lower than on the GPU.

Also, NVIDIA Jetson AGX Xavier has two deep learning accelerators (DLAs), which are slower than the GPU and can execute complete or partial deep learning models. However, although using DLAs increases execution time, they are more energy efficient than the GPU. Moreover, it allows the release of GPU resources for other tasks. DLAs also have a limited functionality to compute DL models compared to GPU. The main limitations of DLAs are: (i) some layers cannot be mapped on Xavier DLAs due to they are not compatible with this hardware,³ e.g. convolutional layers whose kernel size is bigger than 32, softplus layer or softmax layer; (ii) DLA can only map 16 model subgraphs, which are a set of consecutive supported layers. If the number of subgraphs is exceeded, the remaining layers are mapped in the GPU; (iii) DLA can only host one model at a time while GPU can host different models; (iv) numerical precision is currently limited to FP16 and INT8.

5.1.2. Numerical precision

The use of lower numerical representations is especially useful in deep learning models and allows for the reduction of both execution time and energy consumption. Data in these models are originally expressed in FP32 format. To reduce the numerical representation of the models, there exists a set of techniques called quantization which can be applied after [25,55] or during [56,57] model training. Although

³ NVIDIA TensorRT documentation can be consulted to find mapping incompatibilities: <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-841/developer-guide/index.html>.

post-training quantization is easier to implement, it usually results in a degradation of the model accuracy while during-training quantization is more computationally expensive but enhances model performance.

In NVIDIA Jetson AGX Xavier, GPU can deal with FP32, FP16 and INT8 data representations. However, DLA can only cope with FP16 and INT8. To transform our models from FP32 to FP16 representation we apply TensorRT framework, which employs a post-training quantization approach. Nevertheless, INT8 quantization needs an additional calibration to keep accuracy when the network is converted from FP32 to INT8. In order to perform this calibration process, we have fine-tuned the model using PyTorch *quantization aware training*. During this fine-tuning, all calculations are made in floating point, using fake quantization modules. These modules model the effects of quantization by clamping and rounding to simulate the effects of INT8 precision. Finally, the calibrated model is deployed using TensorRT to complete the optimization process. In the case of DLAs, INT8 quantization has not been applied due to layer incompatibilities (as previously commented in Section 5.1.1) with this data precision.

5.2. Pipeline deployment

According to the characteristics of the target hardware, we have made the following decisions regarding the models and techniques employed to implement the pipeline stages presented in Section 4.

5.2.1. Region proposal network (RPN)

We have evaluated several YOLO versions like YOLOv4, YOLOv5, YOLOv6 and two different versions of YOLOv7 used as RPN, however, the accuracy does not increase significantly (48.66%, 49%, 52.8%, 51.2% and 52.9%, respectively) but the inference time on our embedded system does (57.43 ms, 83.61 ms, 96.27 ms, 83.05 ms and 141.02 ms, respectively). Therefore, we employ a YOLOv4 [58] since it provides a good balance between the number of layers, millions of parameters, and the average precision. However, any existing object detection model could be used as RPN according to the application requirements. In order to improve YOLOv4 performance in our embedded system, we have changed the original CSPDarknet53 backbone with ResNet50 [59] since the latter is highly optimized in NVIDIA Jetson devices in terms of energy consumption and inference time. For example, TensorRT selects automatically the appropriate precision in this model, allowing mixed precision between layers. Moreover, TensorRT selects the most optimized kernels for a specific device and configuration, among other things. Note that, although newer YOLO has been released, their complexity is higher which incurs higher execution times on embedded systems. Finally, due to the input shape required by this model being 640×384 pixels, the original video frames have been resized accordingly.

5.2.2. Optical flow region proposal (OFRP)

Since the CPU is not used by our deep learning models and our pipeline does not require extremely accurate optical flow maps to extract the ROIs, we have chosen Farneback Dense Optical Flow technique [60] to deploy this stage but any other technique could be used. In our case, the selected technique can be implemented on the CPU maintaining a good balance between computational complexity and the quality of the calculated optical flow maps. This way, we increase the resource use in the embedded platform. In this case, since the performance of the optical flow depends on the input size, we have selected a resolution of 720×405 pixels after testing different options looking for a compromise between accuracy and computational time.

5.2.3. Non-maxima suppression (NMS)

As the calculations to carry out NMS do not require much computational cost since it just needs to compute the IoU metric between bounding boxes, this stage is executed on the CPU.

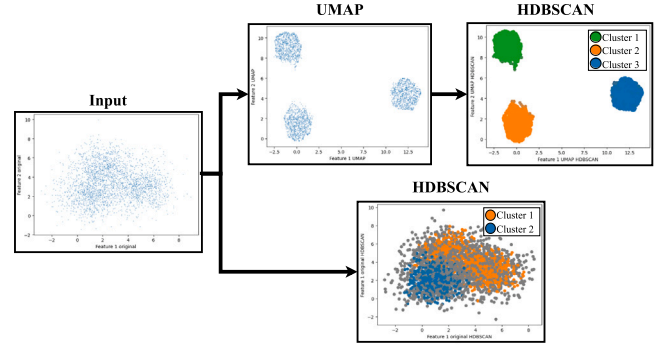


Fig. 5. Functionality of HDBSCAN and UMAP. The upper row shows the use of UMAP (Uniform Manifold Approximation and Projection) dimensionality reduction technique together with HDBSCAN clustering algorithm, obtaining three clearly differentiated clusters. The lower row uses only HDBSCAN algorithm resulting in two confusing clusters. This comparison highlights the effectiveness of employing UMAP in combination with HDBSCAN.

5.2.4. Region encoding network (REN)

In this stage we use a pretrained ResNet-50 model trained on ImageNet [61] with Supervised Contrastive (SupCon) Loss [62] because it provides a more robust clustering of the representation space since it minimizes the intra-class distance and maximizes the inter-class distance between samples. Moreover, among the advantages of SupCon, we can highlight that it is robust to natural corruptions such as noise, blur, or compression, and it is more stable to hyperparameter settings such as optimizers and data augmentations. Since the NMS stage of our pipeline proposes several regions after analysing each frame, the inference on REN processes all detections in parallel.

5.2.5. Clustering (HDBSCAN+UMAP)

In this stage, we have used HDBSCAN algorithm [63] that is able to group similar regions based on features, dealing with different cluster shapes and densities. Furthermore, HDBSCAN provides a sort of probability derived from the distance of a given sample to all clusters. These probabilities allow us to remove wrong detections since we can discern how close is a sample to each cluster and its relationship with other clusters.

To improve the clustering performance, we use UMAP (Uniform Manifold Approximation and Projection) technique [64] for dimensionality reduction since it has been shown that the use of UMAP can drastically improve the performance of well-known clustering algorithms [52]. In Fig. 5 we use a toy example to show the benefits of applying UMAP before HDBSCAN.

6. Experimental results

In this section, we first present the apron area dataset used in our experiments and the implementation details of the proposed pipeline. Next, we define the metrics employed to measure and compare the implementation performance. After that, we carry out a wide range of experiments using baseline models and optimized models, evaluating their impact on energy consumption and inference time. Moreover, we show the effect of our two region proposal algorithms according to their individual CorLoc metrics and we employ mean Average Precision (mAP) in order to evaluate the object classification task. Finally, we propose some guidelines according to the obtained experience.

6.1. Dataset

In order to test our proposed pipeline, we use a video dataset captured by a live RGB camera that is recording permanently the apron area (the area where aeroplanes are parked, loaded, unloaded,

refuelled, maintained, or boarded) of the Gdansk Airport. This camera is publicly accessible online. The dataset contains 96 video clips one minute long recorded with a resolution of 1920×1080 pixels and a frame rate of 15FPS. To deal with different lighting conditions, 60% of the videos were recorded in the morning and the other 40% were recorded during the afternoon/evening. In our experiments we consider the following categories of objects: car ('car'), fire-truck ('ft'), fuel-truck ('fuel'), luggagetrain-manual ('lgm'), luggagetrain ('lg'), mobile-belt ('mb'), person ('pe'), plane ('pl'), pushback-truck ('pb'), stairs ('st') and van ('van'). Note that the abbreviated name of each class used in the tables is included in parentheses. With the aim of assessing the performance of the proposed solutions, we have manually labelled all videos.

6.2. Implementation details

Our optimized YOLOv4 with ResNet50 as backbone has been trained on a computer with a discrete GPU (NVIDIA GeForce RTX 3090) using PyTorch 1.11. Regarding the *REN* model, i.e. ResNet50 with Supervised Contrastive Loss [62], it is implemented using the pretrained weights available in this repository.⁴ As mentioned above, we ran our experiments on NVIDIA Jetson AGX Xavier. Inference processes have been deployed with Python 3.8 using TensorRT 8.4.1 and cuDNN 8.4.1. *RPN* and *REN* have been deployed on only GPU or GPU together with *DLA*, *RPN* on *DLA-0* and *REN* on *DLA-1*. On the other hand, *OFRP*, *UMAP* and *HDBSCAN* are computed on the CPU and are implemented with OpenCV, Umap-learn and Scikit-learn libraries, respectively. Notice that *UMAP* is not implemented on GPU due to library incompatibilities on Jetson AGX Xavier. Finally, non-maxima suppression and data preprocessing before introducing frames into *OFRP* or *RPN* are calculated on the CPU. The complete code of our pipeline will be available at.⁵

Regarding parameters commented in Section 4, after a cross-validation process on a subset of the training data, we have empirically established the following values: $T_F = 1$, $T_A = 66$, $T_I = 0.4$, $T_{AR} = 0.5$. Note that the higher T_F , the more motion would be eliminated, hence, there is a greater possibility of removing valid regions. On the other hand, the lower T_F , the higher the probability that the detected motion is actually noise and not valid regions. For example, in scenarios with subtle but significant motion (e.g. slow moving objects), a low T_F ensures detection, although with potential noise. The same occurs with T_A : if it is too large, we eliminate valid regions, but if it is small, we introduce noisy regions. Finally, T_I and T_{AR} are thresholds that serve to eliminate redundant detections from both the *RPN* and *OFRP*. Consequently, excessively high values may lead to the removal of distinct nearby objects detected by each algorithm, such as when multiple overlapping objects are present in a crowded scene, while excessively low values could result in the duplication of objects detected by both algorithms, increasing post-processing complexity. Therefore, these thresholds are crucial for balancing detection precision and computational efficiency.

6.3. Performance and accuracy metrics

On the one hand, we analyse energy consumption and execution time for every experiment. Energy consumption is given by internal sensors incorporated on Jetson boards. We use a sampling frequency of 500 Hz, which is the maximum value supported by the energy sensors. To ensure the quality of energy and time measurements, we averaged values for 200 forward steps.

On the other hand, to evaluate the performance of our pipeline we use two metrics: Correct Localization (CorLoc) metric and mean Average Precision (mAP). CorLoc [65,66] measures the precision of our

Table 3

Baseline study: Time and energy consumption. Each row represents a different stage: preprocessing (*Prep.*), region proposal network (*RPN*), optical flow region proposal (*OFRP*), non-maxima suppression (*NMS*), Region Encoding Network (*REN*), *UMAP* algorithm (*UMAP*) and *HDBSCAN* algorithm (*HDBSCAN*). Columns represent where the model is going to be deployed, inference time in milliseconds (ms) and GPU or CPU energy consumption in millijoules (mJ).

Stage	Deployment	Time (ms)	Energy (mJ)
<i>Prep.</i>	CPU	3.22	2.69
<i>RPN</i>	GPU	84.89	62.79
<i>OFRP</i>	CPU	67.30	45.13
<i>NMS</i>	CPU	1.12	0.93
<i>REN</i>	GPU	1505.04	5589.54
<i>UMAP</i>	CPU	5725.44	3895.01
<i>HDBSCAN</i>	CPU	45.83	10.82

region proposal method, that is, the accuracy of the region localization as shown in Eq. (3). In this equation N is the total number of objects, P_i represents the predicted region, G_i is the ground-truth region and $\mathbb{1}$ is an indicator function that equals 1 if the condition inside is true and 0 otherwise. The region of an object is correctly localized if the IoU between the predicted region and the ground-truth region is bigger than a threshold τ , which is set as the standard value of 0.5 for the *RPN* and 0.1 for the *OFRP*.

$$CorLoc = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(IoU(P_i, G_i) > \tau) \quad (3)$$

Similarly, mAP measures the object recognition accuracy and it indicates the Average Precision (AP) across all classes as presented in Eq. (4), where C is the total number of object classes in the dataset and AP_c is the average precision for a specific class c , computed as the area under the Precision-Recall curve for that class [67].

$$mAP = \frac{1}{C} \sum_{c=1}^C AP_c \quad (4)$$

6.4. Throughput results

In this subsection, we show a complete evaluation of our proposed pipeline in terms of execution time and energy consumption. We first evaluate our baseline model without any optimization. Then, we apply different optimizations and finally, we present the best pipeline deployment taking advantage of several optimizations.

6.4.1. Baseline

In Table 3, we show the obtained results in terms of energy consumption and execution time for our baseline model without any optimization (TensorRT is not used). Thus, the models (*RPN* and *REN*) have been deployed with PyTorch on GPU using FP32 data representation, and the remaining pipeline stages have been deployed on CPU as previously commented in Section 6.2. Each table row indicates the obtained values for two different metrics (time and energy) per pipeline stage. As it can be observed, the most time and energy-consuming stage corresponds to the *UMAP* algorithm, which consumes 77.03% of the total execution time and 40.54% of the total energy. This way, our complete pipeline, without considering any optimization, takes 7.43 s which is far from real-time execution. Following, we propose a modification of the *UMAP* technique to reduce its execution time.

6.4.2. Improving the clustering stage

In the previous Subsection we discovered that in the initial approach only *UMAP* takes 5.73 s per image which is unfeasible for our real-time application. Therefore, in order to decrease the execution time and maintain the accuracy of the clustering stage, we propose to emulate the *UMAP* algorithm using an easy-to-implement lightweight DL model (*UMAP-M*). After some experimentation, we found that a model

⁴ <https://github.com/HobbitLong/SupContrast>

⁵ <https://github.com/PaulaRuizB/Embedded-object-detection>

Table 4

TensorRT optimizations. Comparison between the original non-optimized model and TensorRT-optimized model for region proposal network (*RPN*), Region Encoding Network (*REN*) and *UMAP* model (*UMAP-M*). Columns represent inference time in milliseconds (ms) and energy consumption in millijoules (mJ) for GPU using FP32.

Stage	Model (P32)	Time (ms)	Energy (mJ)
<i>RPN</i>	Baseline	84.89	62.79
	TRT	57.43	38.23
<i>REN</i>	Baseline	1505.04	5589.54
	TRT	869.44	3501.51
<i>UMAP-M</i>	Baseline	5.39	0.67
	TRT	1.67	0.65

composed of five fully connected layers with 128 neurons was enough accurate. Changing the algorithm by this model can be considered as a kind of knowledge distillation [68], where we transfer knowledge from a slow algorithm that runs on CPU (*UMAP*) to a faster one. We have employed a thousand frames of our dataset to train our *UMAP-M* during 100 epochs with a learning rate of 0.001 using 0.2 dropout and mean squared error loss. We trained our model using one GPU GeForce RTX 3090 with 24 GB in a server composed of an Intel Xeon Silver 4314 processor with 64 cores, running at 2.4 GHz, 500 GB of RAM. As *UMAP-M* is a DL model, its execution will be mapped on GPU. This way, we reduce the execution time for the clustering process from 5.73 s to 5.39 ms when running this model with FP32 on the GPU, which results in an improvement of 1062×, and the energy consumption from 3895.01 mJ to 0.67 mJ, which results in an improvement of 5832×.

6.4.3. TensorRT optimizations

In this Section, we study the impact of including TensorRT optimizations such as kernel autotuning, layer/tensor fusion, and batching in our deployment. Quantization, another important optimization, will be explored in the following subsection. The impact of TensorRT optimization is evaluated by comparing the FP32 baseline implementation with the FP32 optimized version of TensorRT.

Results of conducted experiments are shown in Table 4, where inference time and energy consumption are shown for *RPN*, *REN* and *UMAP-M*. For the case of *RPN*, there is a decrease in inference time of 27.46 ms and a reduction of 24.56 mJ in energy consumption. On the other hand, when *REN* is running using TensorRT its inference time decreases 635.60 ms and energy consumption decreases 2088.04 mJ. Finally, for *UMAP-M* the reduction in inference time is 3.72 ms and 0.02 mJ for the energy consumption. Therefore, the use of specialized frameworks (TensorRT in our case) for DL deployment on embedded devices improves significantly inference time and energy consumption. Note that, this improvement will be more noticeable for large models, as in our case for *REN*.

6.4.4. Quantization

Two data representations are studied in this Subsection, FP16 and INT8. Table 5 shows the achieved results. For the sake of comparison, FP32 results are also shown. Focusing on FP16 data precision, we can see a significant reduction in energy consumption and inference time for *RPN* (third row) and *REN* (sixth row) with respect to FP32. Note that for *UMAP-M* the drop for both FP16 and INT8 is negligible because of the small size of the network. For *RPN* the reduction in energy consumption is around 4.6× and in inference time is around 2.8×. For *REN*, the decrease in energy consumption is around 10.4× and in inference time is around 4.2×. On the other hand, for *RPN*, INT8 precision has a non-expected performance as it takes longer than using FP16. In order to find an explanation for this behaviour, we have analysed the executed kernels with the NVIDIA Profiler⁶ for INT8

Table 5

Quantization. Each row shows models optimized with TensorRT using different data precision (FP32, FP16 or INT8) for each model, Region Proposal Network (*RPN*), Region Encoding Network (*REN*) or *UMAP* model (*UMAP-M*). Columns represent inference time in milliseconds (ms) and energy consumption in millijoules (mJ) for GPU.

Model	Quantization	Time (ms)	Energy (mJ)
<i>RPN</i>	FP32	57.43	38.23
	FP16	20.54	8.36
	INT8	42.66	24.36
<i>REN</i>	FP32	869.44	3501.51
	FP16	208.53	337.83
<i>UMAP-M</i>	FP32	1.67	0.65
	FP16	1.58	0.64
	INT8	1.57	0.64

quantization. We observed that TensorRT uses not only INT8 kernels but also FP32. This happens because TensorRT does not have an INT8 implementation for some *RPN* layers. Therefore, the inference time decrease is not as high as expected since mixed precision, i.e. FP32 and INT8, are used together. Besides that, INT8 quantization for *REN* is not shown because the parameter network calibration required to keep a good accuracy after applying this quantization needed a powerful multi-gpu server which was not available. Just to justify this, similar trainings of ResNet models with batch sizes between 2048 and 6144 for SupCon loss required 8 NVIDIA V100 GPUs [62].

6.4.5. Deep learning accelerators

Our embedded platform includes two identical DLAs that can be used to perform DL models inference. However, unlike GPU, the hardware capabilities of the DLAs have several constraints and, consequently, complex models may not be completely deployed in these accelerators. This is the case of the three models employed in this work: *RPN*, *REN* and *UMAP-M*. Nevertheless, TensorRT is able to partition the models so that some layers are performed in DLA and others in GPU. An aspect to be considered is that GPU and DLA data interchange incurs some overheads due to, among other causes, both devices employing different tensor format shapes. In Table 6 we show the performance results obtained with our *RPN* and *REN* using FP16 data precision. For the sake of comparison, results for the equivalent deployment using only GPU are also indicated.

Regarding the *RPN*, several layers of this model employ Softplus activation function that cannot be mapped on DLA. Consequently, many data interchange between GPU and DLA is carried out during the model inference, incurring a high overhead. It can be observed that GPU+DLA inference is much slower than only GPU. Also, the energy consumption is worse due to the long latency. Referring to *REN*, incompatible intermediate layers cause the overhead in inference time and energy consumption due to data interchange between GPU and DLA. As in the previous model, only GPU implementation is more efficient than GPU+DLA implementation.

Finally, the deployment of *UMAP-M* on GPU+DLA has been discarded because in spite of its small size increases both latency and energy consumption w.r.t only GPU. Thus, in GPU+DLA, DLA performs a matrix multiplication per layer while the previous transposing of one of the matrices must be performed on GPU. Again, as in the *RPN* and the *REN*, data interchange at each layer hurts the performance.

6.4.6. Pipeline deployment

After studying different implementations of the pipeline stages shown in Fig. 3, we proposed in Section 5.1.1 that DL models must be executed on GPU, while the remaining task should run on CPU. Regarding models, it can be also observed that FP16 versions of the models achieved better performance than both FP32 and FP16+DLA versions. Taking into account all this information, we propose a deployment where CPU and GPU tasks are concurrently executed. The task scheduling scheme is shown in Fig. 6. It depicts the processing of

⁶ NVIDIA Nsight Compute documentation can be consulted: <https://developer.nvidia.com/nsight-compute>.

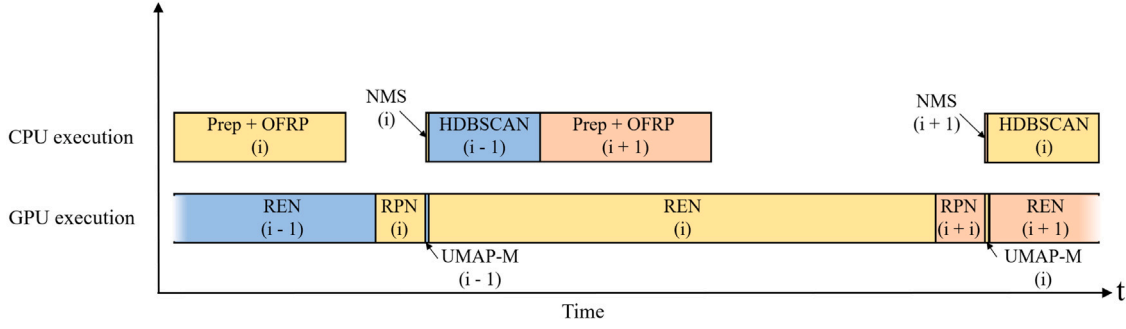


Fig. 6. Concurrent deployment on CPU and GPU. The upper row shows the tasks deployed on CPU, *i.e.* preprocessing (*Prep*), Optical Flow Region Proposal (*OFRP*), non-maxima suppression (*NMS*) and *HDBSCAN* algorithm (*HDBSCAN*). The lower row shows the tasks deployed on GPU, *i.e.* Region Encoding Network (*REN*), Region Proposal Network (*RPN*) and *UMAP* model (*UMAP-M*). Three consecutive samples ($i - 1$, i , and $i + 1$) are captured. Note that pipeline stages processing the same sample are represented with the same colour.

Table 6

Deep Learning Accelerators inference. Each row shows models optimized with TensorRT using FP16 data precision on GPU or GPU together with Deep Learning Accelerator (DLA) for Region Proposal Network (*RPN*) and Region Encoding Network (*REN*). Columns represent inference time in milliseconds (ms) and energy consumption in millijoules (mJ).

Model	Deployment	Time (ms)	Energy (mJ)
<i>RPN</i>	GPU	20.54	8.36
	GPU + DLA	59.89	32.55
<i>REN</i>	GPU	208.53	337.83
	GPU + DLA	932.04	783.44

three consecutive samples ($i - 1$, i , and $i + 1$ samples) captured by a video surveillance camera. Pipeline stages processing a specific sample have been represented with the same colour. Concretely, the stages involved in the complete processing of i sample are shown in yellow. Synchronization primitives have been included so that dependencies among stages processing the same sample are fulfilled. It can be observed that the throughput of the deployment is limited by the tasks running on GPU, that is FP16 versions of *RPN*, *REN* and *UMAP*, which add up to 231 ms (4.3 FPS). This value can be advantageously compared with recent works as [42] that running just the *RPN* in the same embedded device only achieves a bit higher throughput, that is, 5.1 FPS.

6.5. Accuracy related results

In this Subsection, we analyse CorLoc and mAP metrics for our optimized models.

6.5.1. Region proposal comparative

To evaluate the performance of each region proposal algorithm (*RPN* and *OFRP*) we use the CorLoc metric, comparing the region proposals from each approach with the annotated ground-truth. Results for this metric are shown in Table 7 (the higher the better) for our two proposed algorithms. To measure the impact of quantization on the accuracy of the results, *RPN* has been tested using three configurations of the TensorRT models: FP32, FP16, and INT8. Notice that *OFRP* is executed on CPU and, consequently, TensorRT quantization is not applied. Concerning the region proposal algorithms we have three options: only *RPN* (second column), only *OFRP* (third column) or the combination of both (fourth column) using non-maxima suppression stage. As was commented in Section 6.3, the standard value of 0.5 is used as the IoU threshold for the *RPN* algorithm and 0.1 for the *OFRP*. It can be observed that CorLoc results are higher with *RPN* than with *OFRP* as while *RPN* is trained to detect objects, *OFRP* is based on pixel intensities variations which are more prone to noisy locations (e.g. lighting changes). Note that, the improvement in the results for

Table 7

CorLoc results for different data precision. Each row shows the CorLoc results using different configurations: only the Region Proposal Network (*RPN*) in the second column, only the Optical Flow Region Proposal (*OFRP*) in the third column or a combination of both *RPN*+*OFRP* in the last column for different *RPN* data precision. Best result is marked in bold.

Quantization	<i>RPN</i>	<i>OFRP</i>	<i>RPN</i> + <i>OFRP</i>
FP32	8.02	4.06	12.52
FP16	8.03	4.06	12.52
INT8	8.26	4.06	12.92

RPN using INT8 is due to the fact we have fine-tuned the model using PyTorch *quantization aware training*, during four epochs improving slightly the accuracy of the model. Finally, it can be seen in column fourth that the combination of both algorithms shows an important improvement in CorLoc results, proving that the use of the Optical Flow information enhances the capabilities to detect objects. This means that both algorithms are detecting complementary information, *i.e.* different objects.

6.5.2. Class prediction comparative

First of all, to evaluate our pipeline with this metric, it is necessary to assign human-understandable labels to each cluster to match the classes of the annotations in the dataset. However, we want to remark that it is completely optional since our proposal can work without human labels. For doing this voluntary stage, a human observes a set of samples for each cluster obtained by *HDBSCAN*, *i.e.* those samples with the highest probabilities for a given cluster, and assigns a human label to each cluster according to those samples. Noisy clusters with mixed detections are removed if more than half of these detections do not belong to the same cluster.

Therefore, in this experiment, we focus on the class prediction to measure the object classification task. We use *UMAP-M* with *HDBSCAN* to predict the class from the previously obtained bounding boxes and we employ mean Average Precision (mAP) to measure the performance of our pipeline by using labels collected after the clustering process. The obtained results can be observed in Table 8 where mAP considers the objects correctly classified as belonging to five classes: luggage/train-manual, person, plane, fire-truck, and fuel-truck. Each row represents a different experiment taking into account both algorithms previously used for proposing regions, *i.e.* *OFRP* and *RPN* and using different data precision for *RPN*. It can be observed that we obtain more than 50% mAP regardless of the data precision, but best results are obtained with FP32 and FP16 data precision. Fig. 7 illustrates some examples of clustering results. On the one hand, Figs. 7(a) and 7(b) show two examples of correct clusters. Despite variations in background, size, or colour among the samples, the clustering algorithm successfully grouped them. On the other hand, Figs. 7(c) and 7(d) present two clusters with errors. These mistakes could be caused by visual similarities

Table 8

mAP results per data precision for RPN. The mean Average Precision (mAP) is the mean of the Average Precision (AP) for all clustered classes. If no cluster predicts a specific class, it is ignored for the computation of mAP. Best results are marked in bold.

Region Proposal	mAP
RPN FP32 + OFRP	59.44
RPN FP16 + OFRP	59.44
RPN INT8 + OFRP	54.51

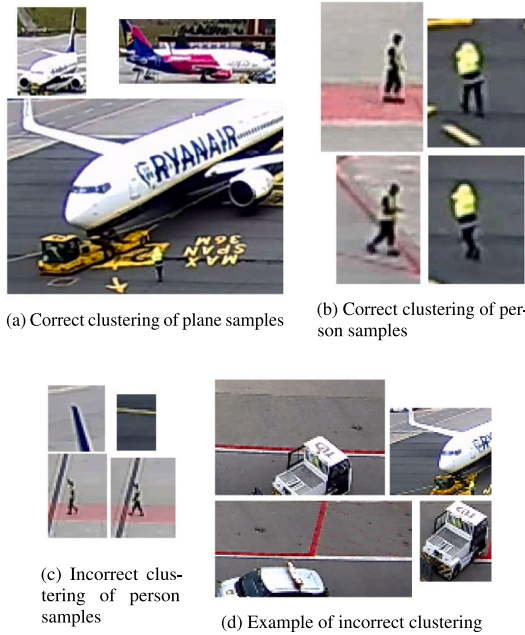


Fig. 7. Clustering results. The top row shows examples of correctly grouped samples, while the bottom row represents incorrectly grouped samples.

in patterns or colours, that challenge the ability of the algorithm to differentiate between objects.

6.6. Final guidelines

In this section, we propose some guidelines, according to the obtained experience, to efficiently deploy deep learning applications on an edge device.

- Although PyTorch or TensorFlow allows us to deploy models on embedded systems, the execution time and energy consumption are higher than using specific deployment frameworks for the target embedded system. Thus, if the manufacturer provides a framework for deploying DL models, it should be used.
- For high-consuming CPU or GPU algorithms, simulating their behaviour with deep learning models, e.g. employing knowledge distillation, could significantly improve the performance of the application.
- Quantization improves energy consumption and inference time without a big impact on the final accuracy. However, if there are no kernels available for a given operation at a given precision, the obtained performance will be worse than expected since those kernels will be executed in FP32.
- In applications sensitive to throughput, such as those that must process video at a specific frame rate, the deployment of tasks must be done in different computing units so that concurrency can help to achieve performance aims.
- The incurred overhead, when complex models are split and executed in different computing units (GPU and DLAs in our case),

can harm the performance by increasing both computing time and energy consumption.

7. Conclusions

In this paper, we have presented a complete unsupervised pipeline for video object detection in real-time using a low-power device, namely Jetson AGX Xavier, together with a set of general guidelines to help other researchers deploy their pipelines on embedded systems. On the one hand, our proposed pipeline, which can be deployed without requiring domain adaptation to the target task, uses RGB and optical flow maps as input. Those inputs are used by the region proposals to detect the objects of the scene. Finally, an unsupervised clustering algorithm is used to group similar detections into virtual classes. This way, these virtual classes can be translated by a human into meaningful labels by visualizing the obtained clusters. Focusing on the accuracy results, the use of both region proposal algorithms always improves both the CorLoc and mAP results. On the other hand, we have carried out a thorough optimization process to obtain the best execution time and energy consumption without penalizing the accuracy of the approach. We have carefully selected the models and algorithms employed in our proposal with the aim of getting the best execution performance taking advantage of all available resources, *i.e.* GPU, CPU and DLAs. This way, we want to remark that quantization reduces execution time and energy consumption and knowledge distillation is able to transform a slow algorithm into a fast model. Moreover, the use of specific accelerators like DLA increases execution time but allows the GPU to release resources for other applications.

In future work, we plan to add incremental learning to our current pipeline with the aim of improving a given model in order to eventually obtain a highly specialized model for a given task. Additionally, we aim to enhance the optical flow region proposal using a deep neural network to improve small objects detection. Scalability will also be addressed by deploying our pipeline on other low-power devices. Finally, we will focus on optimizing real-time performance by applying advanced techniques such as pruning or more aggressive quantization.

CRedit authorship contribution statement

Paula Ruiz-Barroso: Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Methodology, Investigation, Conceptualization. **Francisco M. Castro:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation, Conceptualization. **Nicolás Guil:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work has been supported by the Junta de Andalucía, Spain (P18-FR-3130 and UMA20-FEDERJA-059), the Ministry of Education of Spain (PID2019-105396RB-I00) and the University of Málaga (B1-2022_04).

Data availability

The authors do not have permission to share data.

References

- [1] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: Towards real-time object detection with region proposal networks, *Adv. Neural Inf. Process. Syst.* 28 (2015).
- [2] C.-Y. Wang, A. Bochkovskiy, H.-Y.M. Liao, YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, 2022, arXiv preprint arXiv:2207.02696.
- [3] X. Zhai, A. Kolesnikov, N. Houlsby, L. Beyer, Scaling vision transformers, in: *CVPR*, 2022, pp. 12104–12113.
- [4] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, Swin transformer: Hierarchical vision transformer using shifted windows, in: *ICCV*, 2021, pp. 10012–10022.
- [5] Nvidia jetson AGX orin, 2023, <https://www.nvidia.com/content/dam/en-zz/Solutions/gtc/t21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf> (Accessed 16 September 2023).
- [6] Coral AI, 2023, <https://coral.ai/docs/dev-board/datasheet> (Accessed 16 September 2023).
- [7] Intel neural compute stick 2, 2023, https://www.intel.com/content/dam/support/us/en/documents/boardsandkits/neural-compute-sticks/NCS2_Datasheet-English.pdf (Accessed 16 September 2023).
- [8] Y.-C. Chiu, C.-Y. Tsai, M.-D. Ruan, G.-Y. Shen, T.-T. Lee, Mobilenet-SSDv2: An improved object detection model for embedded systems, in: 2020 International Conference on System Science and Engineering, ICSSE, IEEE, 2020, pp. 1–5.
- [9] Y. Matsubara, M. Levorato, Neural compression and filtering for edge-assisted real-time object detection in challenged networks, in: 2020 25th International Conference on Pattern Recognition, ICPR, IEEE, 2021, pp. 2272–2279.
- [10] G. Wilson, D.J. Cook, A survey of unsupervised deep domain adaptation, *ACM Trans. Intell. Syst. Technol.* 11 (5) (2020) 1–46.
- [11] F.M. Castro, R. Delgado-Escáño, N. Guil, M.J. Marín-Jiménez, A weakly-supervised approach for discovering common objects in airport video surveillance footage, in: Iberian Conference on Pattern Recognition and Image Analysis, Springer, 2019, pp. 296–308.
- [12] Q.-C. Mao, H.-M. Sun, Y.-B. Liu, R.-S. Jia, Mini-YOLOv3: real-time object detector for embedded applications, *Ieee Access* 7 (2019) 133529–133538.
- [13] M. Mohammadi, A. Al-Fuqaha, S. Sorour, M. Guizani, Deep learning for IoT big data and streaming analytics: A survey, *IEEE Commun. Surv. Tutor.* 20 (4) (2018) 2923–2960.
- [14] S.K. Pandey, R.R. Janghel, Recent deep learning techniques, challenges and its applications for medical healthcare system: a review, *Neural Process. Lett.* 50 (2019) 1907–1935.
- [15] S. Liu, L. Liu, J. Tang, B. Yu, Y. Wang, W. Shi, Edge computing for autonomous driving: Opportunities and challenges, *Proc. IEEE* 107 (8) (2019) 1697–1716.
- [16] A. Gholami, S. Kim, Z. Dong, Z. Yao, M.W. Mahoney, K. Keutzer, A survey of quantization methods for efficient neural network inference, 2021, arXiv preprint arXiv:2103.13630.
- [17] T. Liang, J. Glossner, L. Wang, S. Shi, X. Zhang, Pruning and quantization for deep neural network acceleration: A survey, *Neurocomputing* 461 (2021) 370–403.
- [18] TensorRT, 2023, <https://developer.nvidia.com/tensorrt> (Accessed 27 August 2023).
- [19] TensorFlow lite, 2023, <https://www.tensorflow.org/lite> (Accessed 29 August 2023).
- [20] E. Jeong, J. Kim, S. Tan, J. Lee, S. Ha, Deep learning inference parallelization on heterogeneous processors with tensorrt, *IEEE Embed. Syst. Lett.* 14 (1) (2021) 15–18.
- [21] D. Seichter, M. Köhler, B. Lewandowski, T. Wengelfeld, H.-M. Gross, Efficient rgb-d semantic segmentation for indoor scene analysis, in: *ICRA*, IEEE, 2021, pp. 13525–13531.
- [22] X. Xia, J. Li, J. Wu, X. Wang, M. Wang, X. Xiao, M. Zheng, R. Wang, TRT-ViT: TensorRT-oriented vision transformer, 2022, arXiv preprint arXiv:2205.09579.
- [23] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, et al., Tensorflow lite micro: Embedded machine learning for tinyml systems, *PMLR* 3 (2021) 800–811.
- [24] S. Ahmed, M. Bons, Edge computed NILM: a phone-based implementation using MobileNet compressed by tensorflow lite, in: *NILM* 2020, pp. 44–48.
- [25] P. Ruiz-Barroso, F.M. Castro, R. Delgado-Escáño, J. Ramos-Cózar, N. Guil, High performance inference of gait recognition models on embedded systems, *Sustain. Computing: Informatics Syst.* 36 (2022) 100814.
- [26] S. Hossain, D.-j. Lee, Deep learning-based real-time multiple-object detection and tracking from aerial imagery via a flying robot with GPU-based embedded devices, *Sensors* 19 (15) (2019) 3371.
- [27] P. Ruiz-Barroso, F.M. Castro, N. Guil, Real-time unsupervised object localization on the edge for airport video surveillance, in: Iberian Conference on Pattern Recognition and Image Analysis, Springer, 2023, pp. 466–478.
- [28] K. He, G. Gkioxari, P. Dollár, R. Girshick, Mask r-cnn, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969.
- [29] P. Jiang, D. Ergu, F. Liu, Y. Cai, B. Ma, A review of yolo algorithm developments, *Procedia Comput. Sci.* 199 (2022) 1066–1073.
- [30] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, A.C. Berg, Ssd: Single shot multibox detector, in: *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, the Netherlands, October 11–14, 2016, Proceedings, Part I* 14, Springer, 2016, pp. 21–37.
- [31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [32] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: Transformers for image recognition at scale, 2020, arXiv preprint arXiv:2010.11929.
- [33] J. Beal, E. Kim, E. Tzeng, D.H. Park, A. Zhai, D. Kislyuk, Toward transformer-based object detection, 2020, arXiv preprint arXiv:2012.09958.
- [34] Y. Li, H. Mao, R. Girshick, K. He, Exploring plain vision transformer backbones for object detection, in: *European Conference on Computer Vision*, Springer, 2022, pp. 280–296.
- [35] Y. Chen, M. Mancini, X. Zhu, Z. Akata, Semi-supervised and unsupervised deep visual learning: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* (2022).
- [36] B. Pang, Y. Zhang, Y. Li, J. Cai, C. Lu, Unsupervised visual representation learning by synchronous momentum grouping, in: *European Conference on Computer Vision*, Springer, 2022, pp. 265–282.
- [37] C. Vondrick, A. Shrivastava, A. Fathi, S. Guadarrama, K. Murphy, Tracking emerges by coloring videos, in: *Proceedings of the European Conference on Computer Vision, ECCV*, 2018, pp. 391–408.
- [38] Q. Xie, M.-T. Luong, E. Hovy, Q.V. Le, Self-training with noisy student improves imagenet classification, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 10687–10698.
- [39] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *International Conference on Learning Representations*, 2016.
- [40] D. Bokde, S. Girase, D. Mukhopadhyay, Matrix factorization model in collaborative filtering algorithms: A survey, *Procedia Comput. Sci.* 49 (2015) 136–146.
- [41] J. Ren, Y. Guo, D. Zhang, Q. Liu, Y. Zhang, Distributed and efficient object detection in edge computing: Challenges and solutions, *IEEE Netw.* 32 (6) (2018) 137–143, <http://dx.doi.org/10.1109/NET.2018.1700415>.
- [42] C. Choe, M. Choe, S. Jung, Run your 3D object detector on NVIDIA jetson platforms: A benchmark analysis, *Sensors* 23 (8) (2023) 4005.
- [43] X. Yang, Z. Song, I. King, Z. Xu, A survey on deep semi-supervised learning, *IEEE Trans. Knowl. Data Eng.* (2022).
- [44] D. Gudovskiy, S. Ishizaka, K. Kozuka, Cflow-ad: Real-time unsupervised anomaly detection with localization via conditional normalizing flows, in: *WACV* 2022, pp. 98–107.
- [45] D. Zimmerer, F. Isensee, J. Petersen, S. Kohl, K. Maier-Hein, Unsupervised anomaly localization using variational auto-encoders, in: *MICCAI* 2019, Springer, pp. 289–297.
- [46] Y. Li, X. Hu, Y. Zhuang, Z. Gao, P. Zhang, N. El-Sheimy, Deep reinforcement learning (DRL): Another perspective for unsupervised wireless localization, *Ieee Internet Things J.* 7 (7) (2019) 6279–6287.
- [47] M. Chen, K. Liu, J. Ma, X. Zeng, Z. Dong, G. Tong, C. Liu, MoLoc: Unsupervised roaming for device-free indoor localization in a mobile ship environment, *Ieee Internet Things J.* 7 (12) (2020) 11851–11862.
- [48] X. Wang, X. Deng, Q. Fu, Q. Zhou, J. Feng, H. Ma, W. Liu, C. Zheng, A weakly-supervised framework for COVID-19 classification and lesion localization from chest CT, *IEEE Trans. Med. Imaging* 39 (8) (2020) 2615–2625.
- [49] X. Wang, J. Xiang, J. Zhang, S. Yang, Z. Yang, M.-H. Wang, J. Zhang, W. Yang, J. Huang, X. Han, SCL-WC: Cross-slide contrastive learning for weakly-supervised whole-slide image classification, *Adv. Neural Inf. Process. Syst.* 35 (2022) 18009–18021.
- [50] W. Liu, R. Qin, F. Su, Weakly supervised classification of time-series of very high resolution remote sensing images by transfer learning, *Remote Sens. Lett.* 10 (7) (2019) 689–698.
- [51] S. Suzuki, et al., Topological structural analysis of digitized binary images by border following, *CVGIP* 30 (1) (1985) 32–46.
- [52] M. Allaoui, M.L. Kherfi, A. Cheriet, Considerably improving clustering algorithms using UMAP dimensionality reduction technique: a comparative study, in: *Image and Signal Processing: 9th International Conference, ICISP* 2020, Marrakesh, Morocco, June 4–6, 2020, Proceedings 9, Springer, 2020, pp. 317–325.
- [53] F. Jia, D. Zhang, T. Cao, S. Jiang, Y. Liu, J. Ren, Y. Zhang, CoDL: efficient CPU-GPU co-execution for deep learning inference on mobile devices, in: *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, Association for Computing Machinery New York, NY, USA, 2022, pp. 209–221.
- [54] Y. Kim, J. Kim, D. Chae, D. Kim, J. Kim, μ Layer: Low latency on-device inference using cooperative single-layer acceleration and processor-friendly quantization, in: *Proceedings of the Fourteenth EuroSys Conference* 2019, 2019, pp. 1–15.
- [55] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, W. Gao, Post-training quantization for vision transformer, *NeurIPS* 34 (2021) 28092–28103.
- [56] E. Park, S. Yoo, P. Vajda, Value-aware quantization for training and inference of neural networks, in: *ECCV*, 2018, pp. 580–595.
- [57] K. Zhao, S. Huang, P. Pan, Y. Li, Y. Zhang, T. Gu, Y. Xu, Distribution adaptive int8 quantization for training cnns, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, (4) 2021, pp. 3483–3491.

- [58] A. Bochkovskiy, C.-Y. Wang, H.-Y.M. Liao, Yolov4: Optimal speed and accuracy of object detection, 2020, arXiv preprint [arXiv:2004.10934](https://arxiv.org/abs/2004.10934).
 - [59] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
 - [60] G. Farneback, Two-frame motion estimation based on polynomial expansion, in: Scandinavian Conference on Image Analysis, Springer, 2003, pp. 363–370.
 - [61] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, Ieee, 2009, pp. 248–255.
 - [62] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, D. Krishnan, Supervised contrastive learning, *Adv. Neural Inf. Process. Syst.* 33 (2020) 18661–18673.
 - [63] R.J. Campello, D. Moulavi, A. Zimek, J. Sander, Hierarchical density estimates for data clustering, visualization, and outlier detection, *ACM Trans. Knowl. Discov. from Data (TKDD)* 10 (1) (2015) 1–51.
 - [64] L. McInnes, J. Healy, J. Melville, Umap: Uniform manifold approximation and projection for dimension reduction, 2018, arXiv preprint [arXiv:1802.03426](https://arxiv.org/abs/1802.03426).
 - [65] S. Kwak, M. Cho, I. Laptev, J. Ponce, C. Schmid, Unsupervised object discovery and tracking in video collections, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 3173–3181.
 - [66] H.V. Vo, F. Bach, M. Cho, K. Han, Y. LeCun, P. Pérez, J. Ponce, Unsupervised image matching and object discovery as optimization, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 8287–8296.
 - [67] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft coco: Common objects in context, in: *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13*, Springer, 2014, pp. 740–755.
 - [68] J. Gou, B. Yu, S.J. Maybank, D. Tao, Knowledge distillation: A survey, *Int. J. Comput. Vis.* 129 (2021) 1789–1819.
- Paula Ruiz-Barroso:** received the B.Sc. degree in Electronics, Robotics and Mechatronics from the University of Málaga and Sevilla, Spain and the M.Sc. degree from the University of Málaga, Spain. She is currently Ph.D. research student at the University of Málaga. She has worked, as a visiting student, at the ESL of EPFL (Switzerland). Her research interests include embedded systems, deep learning and edge computing.
- Francisco M. Castro:** received the B.Sc. degree in Computer Science from the University of Córdoba, Spain, and the M.Sc. and Ph.D. degrees from the University of Málaga, Spain. He has worked, as a visiting student, at the THOTH Group of Inria (France). His research interests include human detection, gait recognition, human-centric video understanding and machine learning.
- Nicolás Guil:** received the B.S. degree in Physics from the University of Sevilla, Spain, in 1986, and the Ph.D. degree in Computer Science from the University of Málaga, in 1995. He is currently a Full Professor with the Department of Computer Architecture, University of Málaga. He has published more than 80 papers in international journals and conferences. His research interests include the areas of parallel computing and video and image processing.